SOFTWARE ENGINEERING
& PROJECT MANAGEMENT

DAY 14

Requirement Validation, Feasibility Study & Requirements Management

Student Book • Module II: Requirements Engineering

Learning focus
Check that documented requirements describe the right system, assess whether the proposed solution is viable, and manage requirement changes in a controlled and traceable way.

Running case study: College Event Management System (CEMS)

Designed for classroom projection, discussion, practical application, and revision.

1. Learning Outcomes

By the end of this session, you should be able to:

• Explain the purpose of requirements validation and distinguish it from verification.

• Use reviews, prototypes, test-case derivation, and acceptance criteria to validate requirements.

• Describe technical, economic, operational, schedule, and legal/policy feasibility.

• Explain how requirement changes are recorded, assessed, approved, and traced.

• Apply a simple validation checklist and change-control workflow to a case study.

Core idea: validation asks whether the requirements describe the system stakeholders actually need; feasibility asks whether the proposed solution is practical; requirements management controls requirements and changes throughout the project.

2. Verification and Validation

Concept	Main question	Example
Verification	Are the requirements and work products specified correctly and consistently?	Review an SRS for clear wording, unique IDs, consistency, and testability.
Validation	Do the requirements describe the right system for stakeholder needs?	Ask students and coordinators to confirm that the SRS covers their real registration workflow.

Both activities matter. A requirement can be written clearly and consistently yet still describe the wrong feature. Conversely, a useful stakeholder need can be documented so vaguely that a development team cannot implement or test it reliably.

3. Requirements Validation Techniques

3.1 Requirements reviews and walkthroughs

Stakeholders and team members examine requirements together to identify omissions, ambiguity, conflicts, unrealistic expectations, and incorrect assumptions.

• Read requirements one by one and ask what user need each supports.

• Check whether terms, conditions, outcomes, and exceptions are clear.

• Record each issue with an owner, action, and status.

• Revise the requirements and circulate important changes for confirmation.

3.2 Prototyping

A sketch, wireframe, or clickable mock-up helps users react to a visible workflow. For CEMS, a prototype can show event details, capacity status, registration, and confirmation. Ask users to complete realistic tasks and explain where they hesitate.

3.3 Test-case derivation

Try to derive a pass/fail check from each requirement. If the team cannot determine what evidence would demonstrate success, the requirement may be ambiguous or lack measurable criteria.

3.4 Model and scenario validation

Walk through use cases, process diagrams, or scenarios with stakeholders. Check normal paths, alternate paths, error conditions, permissions, and business rules.

3.5 Acceptance criteria review

Agree on the conditions that must be met for a requirement or feature to be accepted. Acceptance criteria should be observable and should avoid subjective wording.

4. Validation Checklist

• Does each requirement support a business goal, stakeholder need, policy, or constraint?

• Are the correct stakeholders represented in the review?

• Is the wording clear, singular where practical, and unambiguous?

• Are normal, alternate, and error scenarios covered?

• Are requirements feasible and consistent with one another?

• Can each requirement be verified using evidence or a defined review method?

• Are important security, privacy, accessibility, performance, and operational needs considered?

• Are assumptions, dependencies, exclusions, and unresolved issues visible?

• Are priorities and approval status recorded?

• Have corrections been re-reviewed by the appropriate stakeholders?

> Validation is iterative. If a review exposes a missing need, return to elicitation or analysis, update the requirement, and validate it again.

5. Worked Example: Validate a CEMS Requirement

Initial statement: “Students can register for events easily.”

Review question	Finding / action
Is the expected behaviour clear?	Not fully. Define what the student must be able to do and under what conditions.
Are key rules present?	Clarify eligibility, event capacity, duplicate registration, and whether sign-in is required.
Can it be tested?	Not as written. Split it into specific requirements and define observable outcomes.
Is usability measurable?	If needed, agree on a usability target and a representative test method.
Refined examples	“The system shall allow an authenticated eligible student to register when capacity is available.” “After submission, the system shall display whether registration succeeded or was rejected.”

The refined statements are more specific, but they still need validation with stakeholders. Analysis improves the wording; validation checks that the improved wording reflects the intended need.

6. What Is a Feasibility Study?

A feasibility study evaluates whether a proposed system is practical and worthwhile before the organisation commits significant resources. It helps decision-makers understand benefits, costs, risks, constraints, and viable alternatives.

Common feasibility dimensions

Dimension	Questions	CEMS example
Technical	Can available technology, skills, infrastructure, and integrations support the system?	Can the team implement secure login, registration, capacity checks, and reporting with available skills?
Economic	Do expected benefits justify development, operation, support, and maintenance costs?	Will reduced manual effort and fewer registration errors justify the cost?
Operational	Will the system fit users’ workflows and be adopted and supported?	Can students and coordinators use it with suitable guidance and support?
Schedule	Can the required scope be delivered within the deadline?	Can essential features be ready before the college fest?
Legal / policy	Can the system meet applicable laws, contracts, privacy, security, and organisational policies?	Are data collection, access, retention, and consent practices acceptable?

Feasibility is not just “Can we code it?”

A technically possible solution may still be too expensive, too late, difficult to operate, or inconsistent with policy. A feasibility study compares realistic options and helps determine whether to proceed, reduce scope, change approach, or stop.

7. A Simple Feasibility Assessment

Use evidence where possible and label estimates as estimates. The following table is an illustrative starting point, not a substitute for a project-specific study.

Dimension	Initial finding	Risk / question	Possible response
Technical	Core web workflow appears achievable.	Concurrent registration and identity integration need proof.	Build a small technical prototype and test capacity handling.
Economic	Manual work may be reduced.	Development and ongoing support costs are not yet estimated.	Prepare a cost-benefit estimate including maintenance.
Operational	Users already manage events digitally in parts.	Adoption and training needs are unknown.	Run user walkthroughs and plan short guidance.
Schedule	A limited first release may fit the event calendar.	Approval and testing time could be underestimated.	Prioritise Must-have scope and add a schedule buffer.
Legal / policy	College policies should guide data handling.	Data retention and role access need confirmation.	Consult the responsible policy or data owner.

Possible outcomes include proceed, proceed with conditions, investigate further, revise scope, choose an alternative, or do not proceed. The decision and rationale should be documented.

8. Requirements Management

Requirements management is the ongoing process of recording, organising, tracking, communicating, and controlling requirements as they evolve. It starts when requirements are first identified and continues through delivery and maintenance.

Typical management activities

• Assign unique IDs and record source, rationale, priority, owner, and status.

• Maintain versions and an approved baseline where appropriate.

• Track dependencies, assumptions, open issues, and decisions.

• Record change requests and assess their impact before approval.

• Maintain traceability to design, implementation, test cases, and releases.

• Communicate decisions to stakeholders and update affected artefacts.

Requirements management does not mean preventing all change. It means making changes visible, evaluated, authorised, and traceable.

9. Change-Control Workflow

A practical workflow for a requirement change is:

1. Raise and record the change request.
2. Clarify the reason and expected benefit.
3. Analyse impact on scope, design, code, data, tests, cost, schedule, risk, and dependencies.
4. Decide to approve, defer, reject, or request more information.
5. Update the requirement and document version if approved.
6. Update traceability and affected plans, designs, and tests.
7. Communicate the decision and verify the change after implementation.

Example change request

Request: “Allow students to cancel event registrations.”

Impact area	Questions to assess
Business rules	Until when can cancellation happen? Are exceptions allowed?
Data / status	Is cancellation recorded as a status? Is the history retained?
Capacity	Does a cancelled seat become available immediately?
Communication	Does the student receive confirmation? Is the coordinator notified?

Impact area	Questions to assess
Reports	Do registration totals distinguish confirmed and cancelled records?
Testing	Which cancellation, deadline, capacity, permission, and regression tests are needed?

10. Versioning, Baselines, and Traceability

Versioning

Versioning records how a requirement document or individual requirement changes over time. A simple document history might record version number, date, author, change summary, reviewer, and approval status.

Baseline

A baseline is an agreed reference version used to control changes. It is not frozen forever; changes to it follow the project's agreed process.

Traceability

Traceability connects requirements to their sources and related project artefacts. It helps teams understand why a requirement exists, how it will be verified, and what could be affected by a change.

Requirement ID	Source	Test / evidence	Change impact
FR-04: reject registration when full	Coordinator workshop	TC-CAP-01	Registration service, capacity logic, user message
NFR-PERF-01: response-time target	IT discussion	TC-PERF-01	Event-list API, UI, test environment
FR-CAN-01: cancellation	Approved change request	TC-CAN-01	Registration status, capacity, reports, notifications

11. Roles and Responsibilities

Role	Typical responsibility
Stakeholder / product owner	Clarifies business need, priority, and acceptance decisions within their authority.
Business analyst / requirements analyst	Elicits, analyses, documents, validates, and traces requirements.
Developer / architect	Assesses technical feasibility, dependencies, and design impact.
Tester / QA	Reviews verifiability and derives test conditions and evidence.
Project manager / team lead	Coordinates change decisions and assesses cost, schedule, and resource impact.
Approver / governance owner	Authorises decisions when required by project rules.

Responsibilities may be combined in a small project. What matters is that decisions have clear owners and that changes are not accepted informally when they affect scope, risk, or commitments.

12. Classroom Activity

Activity: Validate and manage a change

Scenario: CEMS requirements have been reviewed. A new request asks the team to allow registration cancellation and automatically reopen the seat.

• Review the statement: "Students can cancel registrations easily." Identify missing rules.

• Write three clarification questions for the requester.

• Define two refined requirements and one acceptance test for each.

• Assess technical, economic, operational, schedule, and policy feasibility at a high level.

• List the design, data, reports, and tests potentially affected by the change.

• Recommend approve, defer, reject, or investigate further, and explain the rationale.

• Update a small traceability table with the requirement source and proposed test case.

Expected deliverable: a validation checklist, feasibility summary, change-impact list, decision rationale, and updated traceability entries.

13. Knowledge Check

1. What is the main question asked by validation?

Whether the requirements describe the right system for stakeholder needs.

2. Name three requirements validation techniques.

Reviews/walkthroughs, prototyping, test-case derivation, model/scenario validation, and acceptance-criteria review are examples.

3. What is the difference between verification and validation?

Verification checks whether work products are specified correctly; validation checks whether they represent the system stakeholders need.

4. Name five feasibility dimensions.

Technical, economic, operational, schedule, and legal/policy feasibility.

5. Why perform a feasibility study?

To evaluate practicality, benefits, costs, risks, constraints, and alternatives before major commitment.

6. Does requirements management prevent change?

No. It ensures changes are recorded, assessed, authorised, communicated, and traced.

7. What should impact analysis examine?

Scope, design, code, data, tests, cost, schedule, risks, and dependencies.

8. Why use a baseline?

It provides an agreed reference version against which changes can be assessed and controlled.

14. Glossary

Term	Meaning
Validation	Checking whether requirements represent stakeholder needs.
Verification	Checking whether a work product is specified or produced correctly against its criteria.
Feasibility study	Assessment of whether a proposal is practical and worthwhile.
Impact analysis	Assessment of what a proposed change may affect.
Change request	A recorded proposal to add, modify, or remove a requirement.
Baseline	An approved reference version used for controlled change.
Version history	A record of document or requirement revisions.
Traceability	Links between requirements, sources, and related project artefacts.

15. Homework and Deliverables

• Prepare a validation checklist for your SRS and apply it to at least eight requirements.

• Write a feasibility summary covering technical, economic, operational, schedule, and legal/policy dimensions.

• Create one change request and assess its impact on requirements, design, tests, cost, and schedule.

• Build a traceability table linking at least five requirements to their sources and test cases.

• Record a version history with at least two revisions and the reason for each change.

• Write a short recommendation to proceed, proceed with conditions, investigate further, revise scope, or stop.

Next module: Software Design—design concepts, cohesion and coupling, modular design, architecture, object-oriented design, and UML.